

Vehicle Dashboard – AWS EC2 Docker Compose Infrastructure

1. Infrastructure Overview

Component	Description
Cloud Provider	AWS EC2
Instance Type	t2.medium
AMI	Ubuntu 24.04 LTS
Storage	10 GiB EBS volume
Security Group	Ports 22 (SSH), 80 (Frontend), 5000 (Backend), 3307 (MySQL) open
Containerization	Docker + Docker Compose
Repository Branch	feature/docker-compose-aws

2. Project Architecture

Containers (via docker-compose.prod.yml):

- MySQL → Database container (port 3307:3306)
- Backend (Flask Subscriber) → Receives vehicle data & stores in MySQL (5000:5000)
- Frontend (React Dashboard) → Displays vehicle telemetry data (3000:80)

Communication flow:

Local Publisher → AWS EC2 Backend → MySQL DB → React Dashboard

3. Deployment Steps (on EC2)

1. SSH into EC2: `ssh -i 'docker-k8s.pem' ubuntu@`
2. Install Docker & Docker Compose: `sudo apt install docker.io docker-compose -y`
3. Clone Repo: `git clone https://github.com/Angad0691996/my_first_vehicle_dashboard.git`
4. Checkout branch: `git checkout feature/docker-compose-aws`
5. Run the stack: `docker-compose -f docker-compose.prod.yml up -d`
6. Access frontend: `http://:3000`

4. System Verification

Check	Command	Expected Output
Verify MySQL	<code>docker exec -it mysql-db mysql -u vehicledbuser -p</code>	DB + Tables exist
Verify Backend API	<code>curl http://localhost:5000/api/logs</code>	JSON vehicle data
Verify Frontend	<code>http://<EC2_PUBLIC_IP>:3000</code>	Vehicle dashboard UI

5. Screenshots to Include

- ec2-docker-containers.png → Output of `docker ps`
- browser-dashboard.png → React dashboard showing live data

- api-logs-browser.png → JSON data on /api/logs
- mysql-data.png → MySQL table view

6. Key Learnings

- Managing full IoT data pipeline with Docker Compose.
- Cross-container networking between backend, database, and frontend.
- Cloud deployment on AWS EC2 using Docker Compose.
- Debugging environment variables and API routing in multi-container environments.